Java Virtual Machine (JVM)
Delving Deep into its Architecture

A Java Virtual Machine can be thought of as an abstract computer that is defined by certain specifications. The author leads readers deep into the architectural details of JVM to give them a better grasp of its concepts.

A virtual machine, or virtualisation, has emerged as a key concept in operating systems. When it comes to application programming using Java, platform-independent value addition is possible because of its ability to work across different operating systems. The Java Virtual Machine (JVM) plays a central role in making this happen. In this article, let us delve deep into the architectural details of JVM to understand it better.

Building basics

Let us build our basics by comparing C++ and a Java program with a simple diagram (Figure 1). The C++ compiled object code is OS-specific, say an x86-based Windows machine. During execution, it will require a similar OS, failing which the program will not run as expected. This makes languages like C++ platform- (or OS) dependent. In contrast, Java compilation produces platform-independent byte code, which will get executed using the native JVM. Because of this fundamental difference, Java becomes platform-independent, powered by JVM.

Exploring JVM architecture

Fundamentally, the JVM is placed above the platform and below the Java application (Figure 2).

Going further down, the JVM architecture pans out as shown in Figure 3. Now let us look into each of the blocks in detail.

In a nutshell, JVM architecture can be divided into two different categories, the details of which are provided below.

Class loader subsystem: When the JVM is started, three class loaders are used.

a. *System class loader:* System class loader maps the class-path environment variables to load the byte code.
b. *Extension class loader:* Extension class loader loads the byte code from *jre/lib/ext.*
c. *Bootstrap class loader:* The bootstrap class loader loads the byte code from *jre/lib.*

Method area: The method area (or class area) stores the structure of the class once it is loaded by the class loader. The method area is very important; it does two things once the class is stored in this area:

a. *Identification:* All static members (variable, block, method, etc) are identified from top to bottom.
b. *Execution:* Static variables and static blocks are executed after the identification phase, and static methods are executed when they are called out. Once all static variables and blocks are executed, only then will the static method be executed.

Heap area: The heap area stores the object. Object instances are created in this area. When a class has instance members (instance variable, instance method and instance block), these members are identified and executed only when the instance is created at the heap area.

Java stacks

In the Java stack area, two threads (main thread and garbage collector) are always running. When the user creates any new thread, it becomes the third thread (Thread-0). When the user creates any method, it is executed by the main thread, inside a stack frame (Figure 4). Each method gets its own stack frame to execute. The stack frame has three sections – the local variable